

Pre-defense transcript

Md. Sakif Khan

Rehearsal script for **pre-defense-0421052099.pdf** — 22 pages: a title, twenty body slides, a closing slide.

The four backup slides are a **separate file**, **pre-defense-0421052099-backup.pdf**. You do not present them. Open it alongside the main deck and jump to a slide when a question calls for one; the table near the end of this file maps each to its question.

Budget: 24 min 55 s of speaking against a 25-minute limit. That leaves five seconds, and every row below is now set to exactly what its own words take at 93 wpm — so there is no slack on any individual slide either. Both numbers are a constraint, not a cushion.

The table read 24:26 until its rows were checked against the words above them, and fourteen of the twenty-two demanded a faster rate than this script's own — slide 19 worst, at 125 wpm against 93. Every round's additions had been costed against the total and never against the row they landed in, so the table stayed internally consistent while drifting away from the speech it describes. The same words, re-timed, come to 24:55.

Five seconds is not a margin. Everything that could move to the answers has already moved there, and what is left is spoken because it has to be. A real cushion has to come out of what is said: about 60 words buys 40 seconds. The recovery note below names the slides that can give it up — 3, 6 and 8 — and that is a decision about those slides, not about this one.

Times below are *cumulative at the end of that slide*. If you are more than 40 seconds past a marker, use the recovery notes at the bottom.

#	Slide	Slide time	Cumulative
1	Title	0:19	0:19
2	The problem	1:01	1:20
3	Where existing approaches stop	1:01	2:21
4	Research questions	0:52	3:13
5	AGR: an explicit state machine	1:21	4:34
6	Constrained tools	0:45	5:19
7	The Structural Verification Layer	1:55	7:14
8	One claim, three routes	0:54	8:08
9	Environment and question sets	1:05	9:13
10	Making the comparison fair	1:12	10:25
11	Main results	1:19	11:44
12	Accuracy against cost	0:58	12:42
13	RQ1: hop count	1:15	13:57
14	The caveat I want to raise myself	1:16	15:13
15	RQ2: groundedness	1:10	16:23
16	RQ2: what verification contributes	1:50	18:13
17	RQ3: ablation	1:02	19:15
18	The result I did not expect	1:15	20:30
19	The echo attractor	1:41	22:11
20	The benchmark was wrong 57 times	0:50	23:01
21	Contributions and limitations	1:44	24:45
22	Thank you	0:10	24:55

The three **bold** slides are the ones the committee will actually interrogate. If you are running long, take time from 3, 6, and 8 — never from 11, 13, 14, 17 or 18. That is the same list the recovery notes protect, and 14 is on it for a different reason: skipping it hands the clipping issue to them.

1 — Title

(0:19)

Good morning. I'm Sakif Khan. This is my pre-defense on Agentic Graph Reasoning — knowledge graph navigation with verification before the answer is emitted. My supervisor is Dr. Sadia Sharmin.

Don't read the title aloud. It's on the screen.

2 — The problem

(1:01)

Language models answer factual questions fluently. The difficulty is that they answer just as fluently when they do not hold the fact.

That isn't a rhetorical claim — it's measured here. In my own no-retrieval control on WebQSP, the model asserted 661 entities. 179 of them do not exist anywhere in the knowledge graph. That's 27.1 percent, and every one was stated without a hedge.

The reason this is hard to defend against is that fluency and factuality come out of the same mechanism. The output carries no signal about which one you just got.

■ **Beat after “27.1 percent.”** It is the number that sets up the whole talk.

3 — Where existing approaches stop

(1:01)

Four families of approach, and each stops somewhere specific.

Parametric answering uses no external evidence at all. Vector RAG retrieves once and then reasons — so retrieval happens *before* reasoning begins, and a multi-hop question has to be answered from whatever that single query returned. Static GraphRAG fixes a neighbourhood radius before the question is even understood. Agentic navigation does interleave retrieval and reasoning, which is the right move.

But all four share one gap. Whatever the final generation call produces is what gets emitted. Nothing checks what the answer *asserts* before it is delivered.

■ *Walk the table left to right with the pointer. Don't read the cells verbatim.*

4 — Research questions

(0:52)

Three questions.

RQ1: does agentic navigation actually improve multi-hop accuracy — and does the advantage grow with the number of hops?

RQ2: given a system that already navigates the graph, what does a claim-level check against the traversed triples contribute?

RQ3: which components earn their cost, in accuracy and in tokens?

One framing note. RQ2 asks what verification *contributes*, not whether it reduces hallucination. That's deliberate. The answer has several parts, and a yes-or-no question would have hidden most of them.

5 — AGR: an explicit state machine

(1:21)

AGR is an explicit state machine — not a prompt loop. Six nodes.

The planner decomposes the question into ordered sub-objectives. The explorer scores candidate edges and expands the frontier. The evaluator decides whether the sub-objective has been met. The backtracker undoes a bad expansion and bans the edge that caused it, so it can't re-take the same wrong turn. The verifier is the contribution; more on it shortly. The answerer emits only what survived.

There are three cycles — the three arrows going back to the Explorer: continue, backtrack, retry. All three are bounded by explicit budgets rather than by model behaviour. That gives a termination guarantee that doesn't depend on the model cooperating: every cycle passes through a router that checks a monotone counter.

Do not say “exactly two cycles.” The diagram has three arrows returning to the Explorer and the audience is looking at it while you speak. The thesis caption says two and names two, but the figure source calls the third one a cycle in its own comment. Naming them after the edge labels — continue, backtrack, retry — means counting the arrows confirms the sentence.

If asked about budgets, go to backup page 2.

6 — Constrained tools

(0:45)

The agent never writes a graph query. It gets four operations with fixed signatures — relations, neighbours, an adjacency check, and entity linking.

This matters for more than safety. Because every operation is deterministic and logged with its arguments and result, the traversal is a record. And that record is what the verification layer checks against. Without deterministic tools there would be nothing to verify *against*.

7 — The Structural Verification Layer

(1:55)

This is the contribution.

Before anything is emitted, the draft answer is split into atomic claims. Each claim is checked against the triples the agent actually traversed. A claim the traversal itself grounds keeps those triples and carries them back with the answer. Claims that can't be grounded either trigger targeted re-exploration, or are dropped.

The consequence is that the system hedges rather than asserts, and the answer comes back with its evidence attached.

Three things I want to be precise about. First, why *structural* — the check is against the graph the agent walked, not the model’s opinion of its own output. A model grading itself is not an independent check.

Second, “its evidence” is narrower than it sounds, and the slide says so: one of the three routes records evidence, and the log keeps the count, not the triples. Both come back on slide 15.

Third, the honest limit: a claim can be perfectly true and still be the wrong answer. Structural grounding cannot catch that. I’ll show you exactly that failure in a few minutes.

■ The last sentence is a promise. Slide 19 pays it off.

8 — One claim, three routes

(0:54)

This is the path a single claim takes. Three routes to being called supported.

First, are both endpoint names things we saw during traversal. Second, are they adjacent in the traversal record. Third — and only if the first two fail — an entailment check.

The point of the ordering is cost. Only the third route spends a model call. The first two are pure lookups against a record we already have.

And the “+ evidence” label sits on the first route only.

9 — The environment and the question sets

(1:05)

The environment is a Freebase-derived graph: 2.6 million entities, 8.3 million triples, seven thousand distinct relations. It imports in 36 seconds, which matters only because it means the whole thing is reproducible on one machine.

Questions: 400 each from WebQSP and ComplexWebQuestions. The important column is reachability — for every question I verified whether the gold answer is actually reachable in this graph. 97 percent on WebQSP, 99.2 on CWQ.

That number is the *ceiling* on every accuracy figure I’m about to show. It means when a system misses, the miss belongs to the system and not to the environment.

10 — Making the comparison fair

(1:12)

Five systems: a parametric control, vector RAG, static GraphRAG, Think-on-Graph as the agentic comparison, and AGR.

All five run on one frozen backbone, at temperature zero, under the same 25-call budget, on the same questions, against the same graph.

That's what lets me attribute differences to architecture rather than to model capacity or to spend. One thing is *not* equal, and the slide says so: Think-on-Graph prunes from a narrower candidate set. That cuts against it, not for it — a thinner set is cheaper.

These benchmarks predate current models and may be partly memorised. Without a no-retrieval control I couldn't separate what retrieval contributes from what the model already knew.

Say the width line, don't skip it. The slide claimed a retrieval-budget control it does not have; the Think-on-Graph baseline section names the widths as the one place a reader should look first for a confound, and the conclusion ranks it limitation 5. The numbers are on the slide and in the answer below — you do not have to recite 40/20/300/200 here.

11 — Main results★

(1:19)

Here is the comparison.

AGR reaches 0.755 Hits@1 and 0.642 F1 on WebQSP, and 0.522 and 0.469 on ComplexWebQuestions. That's ahead of every baseline on both datasets.

Two things I'd draw your attention to beyond the top line.

First, vector RAG on ComplexWebQuestions — 0.203, *below* the no-retrieval control at 0.307. One verbalised triple cannot contain a chain, so single-shot retrieval is worse there than not retrieving at all. GraphRAG is beside it at 0.205, but its one-hop radius confounds the paradigm, so the claim rests on vector RAG.

Second, the cost columns. AGR spends 4,511 tokens and 6.2 calls against Think-on-Graph's 3,615 and 12.8. More tokens, but half the calls. I'll come back to why calls are the number that matters.

Slow down here. This is the slide they read while you talk.

Do not pool the two retrieval baselines. The table puts 0.203 and 0.205 side by side and the pooled version of this point is the one that gets walked back: results.tex calls GraphRAG the weaker evidence of the two and says the claim rests on vector RAG. The paper retracted the pooled claim in its own words. If they press on GraphRAG, the answer below has the strata.

12 — Accuracy against cost

(0:58)

Plotted, the frontier depends on which cost you charge, and the two costs I record disagree.

On tokens — the axis here — Think-on-Graph is *not* dominated. It buys lower accuracy at a genuinely lower token price, and any honest reading has to say so. The interior point on WebQSP is static GraphRAG, which the parametric control beats on both axes at once.

On calls, which is what the budget actually meters, Think-on-Graph is dominated on both datasets.

I'm showing both because stating it on one axis alone would overclaim.

13 — RQ1: accuracy against hop count ★

(1:15)

This is the direct answer to RQ1, and it's a shape rather than a difference of means.

Look at ComplexWebQuestions on the right. AGR goes 0.46, 0.55, 0.57 as questions get harder — one hop, two hops, three or more. It is the only system on that dataset that ends above where it started.

Three of the other four decay monotonically. Think-on-Graph is the exception to the monotonicity but not to the direction — it falls and partially recovers, still 0.08 below its own one-hop score.

I want to be careful about the left panel. On WebQSP the three-or-more stratum has n equals 4. I don't draw a conclusion from four questions, and the thesis doesn't either.

Volunteering the $n=4$ weakness pre-empts the obvious attack.

14 — The caveat I want to raise myself

(1:16)

I want to raise this before you do.

If you split the questions by whether Think-on-Graph finished inside the shared 25-call cap, then on the questions it *finishes*, Think-on-Graph is ahead of AGR. 0.852 against 0.788 on WebQSP. 0.629 against 0.607 on CWQ.

The entire aggregate margin comes from the questions it cannot finish — where it drops to 0.197 and 0.188, because it is cut off mid-search. It gets clipped on 29 percent of WebQSP and 44 percent of CWQ.

So the honest claim is not that AGR reasons better per step. It's that AGR completes its reasoning inside a fixed budget, and Think-on-Graph frequently does not. That's an efficiency claim, and it's the one I'm making.

Deliver this as a strength. It is the most defensible slide in the deck.

15 — RQ2: does anything ungrounded get asserted? (1:10)

RQ2. Does anything ungrounded get asserted?

Pooling both datasets: AGR asserts 1,709 entities and zero of them are absent from the graph. The parametric control asserts 1,001 and 22.1 percent of them don't exist.

That looks like a headline result for the verification layer. It isn't — and this is the finding I think matters most.

Think-on-Graph also reaches 0.0 percent. It has no verification layer at all.

So zero ungrounded assertion is a property of *navigating a graph* — if you can only name entities you actually visited, you can't invent one. It is not a property of my verification layer, and I don't claim it as one.

16 — RQ2: so what does verification actually contribute? (1:50)

Which leaves the real question.

What it does not do: removing it changes accuracy by an amount I cannot detect. p equals 1.0 on both datasets. It does not earn its place on Hits@1 or F1, and I report that as a negative result.

What it does do: it withholds what it cannot ground. Removing the layer drops the hedge rate 23.2 to 20.2 percent on CWQ, 8.5 to 8.0 on WebQSP — direction only; I have not tested that column. It attaches supporting triples to the claims traversal grounds, and pairs the answer with that evidence at emission.

The layer's case rests on auditability, not accuracy — and I'll be as plain about how far that goes. Two bounds, both on the slide. One route of three records evidence; `verify_connection` and entailment accept with nothing attached. And the logger writes the *count* of supporting triples and drops the list. You can confirm these answers came from a system that tracked its evidence; you cannot open the record and inspect it.

17 — RQ3: what each component earns ★ (1:02)

RQ3, and this is a four-condition ablation with paired McNemar tests against the full system on stratified halves.

Read the p-value columns. Backtracking: 0.727 and 1.0. Verification: 1.0 and 1.0. Learned scoring versus embedding-only: 0.664 and 0.481. For three of the four components I detect no accuracy effect at this sample size.

I want to be careful with that phrasing — that is “no detectable effect at n equals 200,” not “confirmed no effect.” These are underpowered for small differences and I say so.

One row is different. Removing the planner, on WebQSP, p equals 0.006.

Pause on the 0.006. Then turn the slide.

18 — The result I did not expect

(1:15)

Removing the planner *improves* WebQSP. Plus 0.083 F1, at p equals 0.006, while cutting tokens 31 percent and calls from 6.2 to 4.0. Better, cheaper, and faster, by deleting a component I built.

On ComplexWebQuestions it trends the other way — minus 0.047 F1 at p equals 0.088. Not significant, but the opposite sign, and that’s the useful part.

The explanation is that WebQSP is predominantly one hop. Decomposing a one-hop question invents a second sub-objective that sends the frontier away from the answer it already had.

So the conclusion is that decomposition should be gated on question structure rather than applied unconditionally. That’s a design conclusion the measurement forced on me, not one it confirmed.

Own this. A negative result you can explain is stronger than a positive one you can’t.

19 — Every failure, read: the echo attractor

(1:41)

I read all 259 remaining failures and labelled them. Top categories, pooled.

The one I want to name is sixth, with 13 cases.

The characteristic error of a graph navigator is not invention. It’s what I call the **echo attractor** — the system returns a real, grounded entity that sits one hop from the correct answer. Ask for a director and get the film. Ask for a capital and get the country.

Here’s why it matters, and this is the promise I made on slide seven: any grounding check *passes* it. The entity is real, it was traversed, the triple exists. It is true and it is wrong. Verification cannot catch this by construction.

Different systems fall into it together, so no evaluation treating them as independent can see it. Rescore whenever a majority agree — a natural thing to want — and this becomes apparent correctness. That is the contribution: the mechanism, not the count.

Do not say “it appears across systems, so it is a property of the task rather than of AGR.” That is defensive where the thesis is substantive, and nothing on the slide blames AGR for it. The claim is about evaluation: sec:echo calls the attractor “invisible to any evaluation treating systems as independent”, and section 1.6 says the contribution is the mechanism “and what it means for consensus-based evaluation, not the frequency”. Slide 20 is the same finding seen from the other side.

The table is pooled, and the slide says so. Wrong and hedge are never pooled in the thesis — sec:taxonomy: “a pooled percentage would describe neither” — and pooling also hides the shape flip: `composite_claim` is 1 on WebQSP against 46 on CWQ. The caption carries both facts and points at backup page 4, so the split census is something you offer rather than something you are corrected with.

The full 12-category histogram is backup page 4 if anyone wants it.

20 — The benchmark was wrong 57 times

(0:50)

Same pass, same cross-system agreement: consensus flagged 105 questions, adjudication confirmed 41 — and the gap is the attractor I just described, not label noise.

Those 41 were excluded before the census. 17 more turned up inside it, one is in both, so 57 distinct questions where the benchmark was wrong, not the system.

Published defect rates for these benchmarks are rare and usually anecdotal. This is a counted rate with a documented adjudication procedure and per-question provenance.

21 — Contributions, and what I would not claim

(1:44)

To summarise. Six contributions, and they are the six the thesis claims in section 1.6. The one I would underline is stratum-dependent decomposition: the literature treats decomposition as straightforwardly beneficial, and it is not.

The sixth is worded *pre-registered* in the thesis; nothing was filed with a registry, so I say *pre-specified*.

And the limitations, which I’d rather state than be asked.

The first is the most serious. The verifier persists only what it *rejects*, so wrongful acceptance has no rate at all — and the same decision is why the output contract cannot be audited from the record.

Then: no detectable accuracy gain from verification, and at n around 200 that is “not detected”, not “none”. Zero ungrounded assertion comes from navigation, not my layer. One environment, one backbone, one annotator. And Think-on-Graph leads where it finishes, and prunes from a narrower candidate set — 40 and 20 against my 300 and 200 — so its unclipped subset understates it.

22 — Thank you

(0:10)

Thank you. I’m happy to take questions.

Backup slides — `pre-defense-0421052099-backup.pdf`

A separate document. Open it before you start, minimised or on a second screen. **Everything in this script refers to backup slides by the page number the viewer shows**, which is what this table lists. The file opens on a title page, so the first backup slide is page 2. An ordinal would resolve one short of every one of them: counted that way, the fourth backup slide is hedging rather than the census.

Page	Contents	Use when asked
2	Budget configuration and enforcement sites	“How do you guarantee termination?”
3	Which budgets actually bind	“Is the 25-call cap fair to Think-on-Graph?”
4	Full 12-category failure histogram	“What were the other failure modes?”
5	Hedging rates, all five systems	“Doesn’t it just refuse more often?”

Page 3 is the important one. It shows AGR never reaches the call cap — 0.0 percent on both datasets — which is precisely what makes the comparison against a clipped Think-on-Graph legitimate rather than an artefact of a cap chosen to suit AGR.

Anticipated questions

“Isn’t the 25-call cap arbitrary, and doesn’t it favour AGR?” It’s the cap Think-on-Graph’s own paper operates under. And AGR never reaches it — zero percent on both datasets (backup page 3). If I raised the cap, AGR’s numbers would not move; Think-on-Graph’s would. I say that in the thesis rather than leaving it for someone to find.

“Do the categories look the same on both datasets?” (*Slide 19 is pooled.*) No, and that is the more interesting answer. The census is reported split in the thesis and never pooled, because wrong and hedge describe different failure semantics and the proportions differ sharply by dataset. The clearest case is `composite_claim`: 1 on WebQSP against 46 on CWQ. WebQSP questions mostly are not compound, so the category barely exists there; on ComplexWebQuestions it is the largest single failure mode. A pooled percentage would describe neither dataset. The full split is backup page 4.

“Doesn’t GraphRAG show the same thing?” (*Slide 11 puts 0.203 and 0.205 side by side. Do not pool them.*) No, and the thesis says so rather than letting the two numbers be read together. GraphRAG retrieves a one-logical-hop neighbourhood, so its fall on CWQ confounds static retrieval with a radius I chose in advance. The evidence that it is the radius is in the strata: GraphRAG scores 0.44 on WebQSP’s two-hop questions, which are largely mediator paths a one-hop expansion does reach, against 0.16 on the CWQ two-hop stratum, where the chains are genuine compositions it cannot reach at all. A retriever failing uniformly on depth would not show that gap. There is a second bound on it: it takes at most 100 edges per topic entity with no ordering imposed, and on 72.5 percent of questions at least one topic entity exceeds that degree, so on those it answers from an arbitrary sample. Vector RAG carries the paradigm claim, because one verbalised triple cannot contain a chain at any radius.

“Did both systems see the same candidate sets?” (*The sharper form of the cap question. Slide 10 raises it deliberately — answer it, don’t deflect.*) No, and it is the one thing I do not hold constant. Think-on-Graph keeps 40 relations per entity and 20 neighbours per relation — the pruning widths of the algorithm as published — where AGR keeps 300 and 200. Measured over the committed tool logs, the 40-relation cut binds on 31.6 percent of the 1,651 entities it expanded and the 20-neighbour cut on 32.8 percent of its neighbour calls; AGR’s relation cap binds once in 3,097 expansions and its neighbour cap on 3.3 percent of calls. So the cut is real and it is asymmetric. What it cannot do is rescue the budget argument: a narrower candidate set makes each step *cheaper*, so it cannot explain why Think-on-Graph runs out of calls. What it does mean is that the residual gap on the questions it *finishes* is measured against a system searching a thinner pool, so that figure is a lower bound on what it could resolve at equal width — not an estimate of it. Re-running it at 300 and 200 is the first item in my future work, and it is limitation 5 in the conclusion.

“If verification doesn’t improve accuracy, why keep it?” Because accuracy was never the only claim. It converts silent error into an explicit hedge, and it pairs the answer with the triples that ground it at the point of emission. I report the null on accuracy rather than hiding it — and I report the bounds on the auditability too, which is the next question.

“Show me one supporting triple, then.” (*Expect this. Slide 15 invites it.*) I can’t, from the committed record, and that is a limitation rather than an evasion. `RunLogger` writes `n_supporting_triples` — an integer — and discards the list, so no committed artifact in this work contains a single supporting triple; every statistic I quote about them comes from that counter. What I can show is the traversal record each answer was produced against. Two things follow and I state both: the pairing is real inside the run and unavailable afterwards, and one polarity of the verifier’s error is therefore unmeasured. Persisting accepted claims with their matching triples is one logging change, and it is the first item in my future work. I did not make it late because it would separate the code from results already frozen against it.

“Why isn’t the five-system comparison one of your contributions?” Because the thesis does not count it as one, and slide 21 is the thesis’s list — section 1.6, one subsection per item. The comparison and the hop-count shape are results the contributions rest on; they get slides 11 and 13, which is where the weight belongs. An earlier version of that slide promoted both to contributions and dropped the ablation, the decomposition finding and the protocol to make room — still saying “six”. The thesis had already been audited for exactly that mismatch, where its conclusion counted four against section 1.6’s six.

“How many questions is that hedge difference, and is it significant?” Six, on CWQ: 23.2 percent of 198 against 20.2. The sets nest — there is no question the ablated run hedged on that the full system asserted on — so those are exactly the six the ablated run answered and the layer declined to. None of the six came back correct: all six were assertions that would have been wrong, which is the mechanism, seen at the only place the design isolates it.

On WebQSP it is one question, and there the ablated run got it right. So across the 398 paired questions correctness moved twice, once each way. I would rather say that than be shown it. No, it is not significance-tested; the ablation’s McNemar test is on correctness, not on the hedge column, and I claim the direction and nothing more. *Do not* offer the no-retrieval contrast here. Its 12.2 percent is a hedge rate, not an error rate — backup slide 5 has it in a column headed “WebQSP hedge %” — and AGR hedges *less* than it does, 8.2 against 12.2, so that comparison argues the opposite of what it looks like. No-retrieval’s actual error rate is 170 wrong out of the 351 questions it asserts on.

“Does every accepted claim get evidence attached?” No — one route of three does. Traversed adjacency attaches every traversed triple joining the pair; `verify_connection` and the entailment fallback accept with nothing attached. On the 80-question development set, 13 answers carry no supporting triples: ten are hedges that asserted nothing, one had every claim rejected, and two asserted a single claim certified by the route that records no evidence. Median across all 80 is 3, mean 4.1, maximum 16 — read from the counter, since the triples are exactly what the log does not keep.

“Your planner result says your own design is wrong.” It says the planner is wrong *for one-hop questions*, and WebQSP is mostly one-hop. On ComplexWebQuestions the effect reverses in sign. The conclusion is that decomposition should be gated on question structure — which is a finding, and one I’d have missed without the ablation.

“n=4 in the three-hop WebQSP stratum is meaningless.” Agreed, and I don’t draw a conclusion from it. The hop-count claim rests on ComplexWebQuestions, where the strata are 137, 211, and 49.

“How do you know the gold answers are reachable?” Verified per question against the graph before scoring — 97.0 percent on WebQSP, 99.2 on CWQ. It’s the ceiling on every Hits@1 I report, and it’s stated as such.

“Where do the topic entities come from — doesn’t the system have to find them first?” They are given by the datasets. `use_gold_entities` is on for every run I report, so the question’s annotated mentions go to `search_entity`, which resolves each one to a graph node through the three-stage resolver. Mention *detection* is assumed; mention-to-node resolution is not — that part the system does. It is limitation 7 in the conclusion: the accuracies I report presume a linking step a deployed system would have to perform, and I do not measure that step. What it is not is a between-system confound. The three systems that touch the graph — AGR, Think-on-Graph and GraphRAG — all seed from the same annotated mentions, and neither the parametric control nor Vector-RAG ever sees them.

“Nine of your failures are one bug. Isn’t the census measuring your implementation?” In part, and I would rather say which part. Nine of the 38 `decomposition_error` cases carry the subtype `extraction_bug`, and they share one mechanism: the evaluator resolves every gold value, the drafted sentence names them verbatim, and `answer_entities` then collapses to the sentence’s grammatical subject. WebQTest-1215 drafts all six of Stephen Covey’s professions and scores [`’Stephen Covey’`]. The reasoning was complete and the verifier certified it; what failed is the step that reads entities back out of the draft. That is limitation 8, and the direction is the part worth saying: it depresses my *own* reported accuracy, so on that question shape the headline numbers are a floor rather than an estimate. The fix is an instruction to the claim decomposer, not an architecture change, and it is the first of the three repairs the census earned. The category split is backup page 4.

“Is this reproducible?” Every number in the thesis is generated from frozen run records by a script; none is transcribed. Same for the three data figures in this deck — they are pulled directly from the thesis’s own generated sources.

Recovery notes

If you hit **11:55 (slide 12) more than 40 seconds late**, compress as follows.

- Slide 16 — cut to: *“It doesn’t improve accuracy. It converts silent error into an explicit hedge and attaches evidence. The case is auditability.”* Saves ~35 s.
- Slide 20 — cut to: *“Reading every failure also found 57 questions where the benchmark was wrong, with documented provenance.”* Saves ~25 s.
- Slide 3 — name only the fourth row and the gap. Saves ~30 s.

Never compress 11, 13, 14, 17, or 18. Slide 14 in particular: skipping it means a committee member raises the clipping issue instead of you, and it lands very differently that way.

Delivery

- **Say the number, then what it means.** Not the reverse. “Twenty-seven percent of asserted entities don’t exist — the model is confidently inventing.”
- **Three slides are negative results** (16, 17, 18). Deliver them at normal pace, not apologetically. A student who reports a clean null is more credible than one who reports only wins.
- The word is **hedge**, not “refuse.” A hedge is a calibrated non-assertion.
- If you lose your place, the takeaway bar at the bottom of the data slides is your prompt — read it aloud and continue.